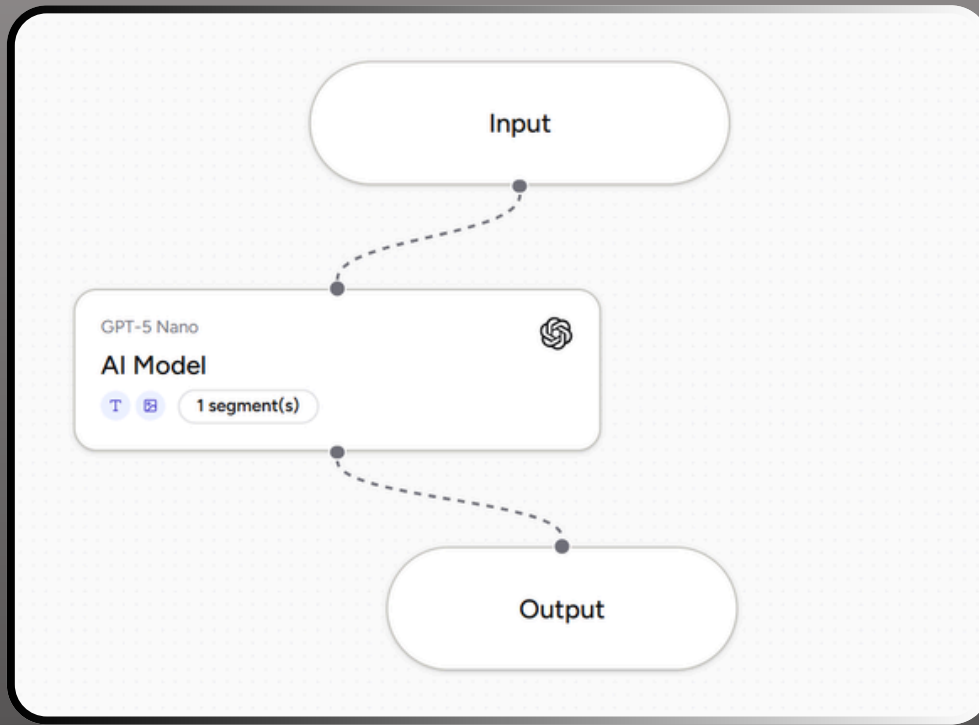




Easy Cybersecurity Home Lab  
using Python, Virtual machines  
and AI.

Slide Next

The image shows a man in a grey hoodie sitting at a desk with a laptop. Overlaid on the image is a complex diagram of a 'SOC Analyst AI Agent' architecture. The diagram includes several components:

- ATTACKER MACHINE**: A terminal window showing network traffic.
- INTERNAL SERVER RUNNING PYTHON AUTOMATION**: A terminal window showing Python code for network automation.
- PUBLIC**: A box representing the public internet.
- AI Model**: A central box labeled 'GPT-5 Nano AI Model' with a '1 segment(s)' indicator, which is part of a larger workflow from 'Input' to 'Output'.

```
import subprocess
import csv
import json
import os
import uuid
import requests
from collections import Counter

# -----
# CONFIGURATION
# -----

INTERFACE = "eth0"          # Channel
CAPTURE_DURATION = 300      # 5 minutes
THRESHOLD = 30              # Packets

PCAP_FILE = "/tmp/traffic.pcap"
CSV_FILE = "/tmp/traffic.csv"
ALERT_FILE = "/tmp/alert.json"
```

# SOC Analyst AI Agent



# Step 1

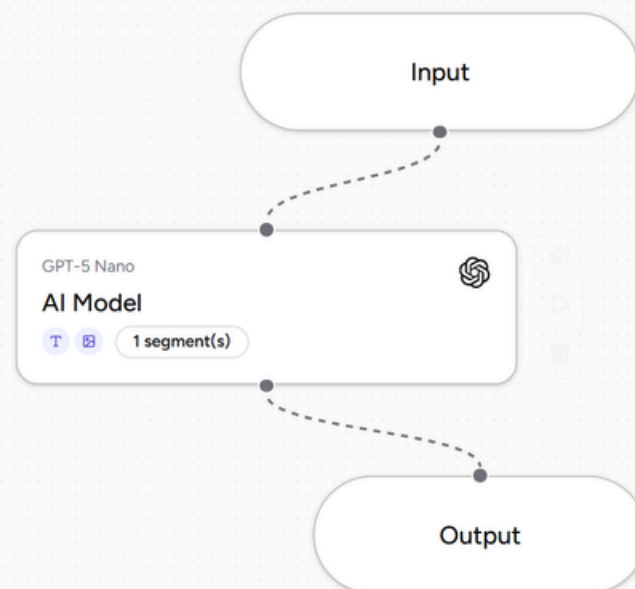
## Build the AI agent

This is the first part of this home-lab. You need to build your own AI agent on airia.ai. The SOC playbook is provided, use it to train your agent.

- Sign up to airia.ai and create your own project

- Add your own AI model and build your agent

- Publish the agent and get the API details including the key



### Junior SOC analyst

Agents

Models

Data Sources

Prompts

User Prompts

Memories

Knowledge Graphs

MCP & Tools

Q Search

Name

Project

Interfaces

Behaviors

 Junior SOC analyst

 Junior SOC analyst



```
import subprocess
import csv
import json
import os
import uuid
import requests
from collections import Counter

# -----
# CONFIGURATION
# -----

INTERFACE = "eth0"           # Change if needed (check with: ip a)
CAPTURE_DURATION = 100       # 5 minutes
THRESHOLD = 40               # Packet threshold

PCAP_FILE = "traffic.pcap"
CSV_FILE = "traffic.csv"
ALERT_FILE = "alert.json"

# ---- Airia Webhook ----
AIRIA_API_URL = "INSERT AIRIA API URL HERE"
AIRIA_API_KEY = "INSERT AIRIA API KEY HERE"
```

The python script has a custom logic built in to find malicious IP's. You can tweak this logic to make it more complex.

## Step 2

# Run the python code

In this step we will write a Python automation script and run it on the Internal server. This script will run a packet capture in the background.

The aim is to find malicious IP's and send those in the form of JSON alerts to AIRIA using the API

```
(royden@kali)-[~/Downloads/soc_analyst]
$ python3 new_soc_capture.py
[+] Capturing on eth0 for 100s
** (tshark:271558) 23:57:24.471605 [WSUtil WARNING] ./wsutil/filter_files.c:242 -- read_filter_list(): '/usr/share/wireshark/cfilters'
line 1 doesn't have a quoted filter name.
** (tshark:271558) 23:57:24.471838 [WSUtil WARNING] ./wsutil/filter_files.c:242 -- read_filter_list(): '/usr/share/wireshark/cfilters'
line 2 doesn't have a quoted filter name.
Capturing on 'eth0'
```

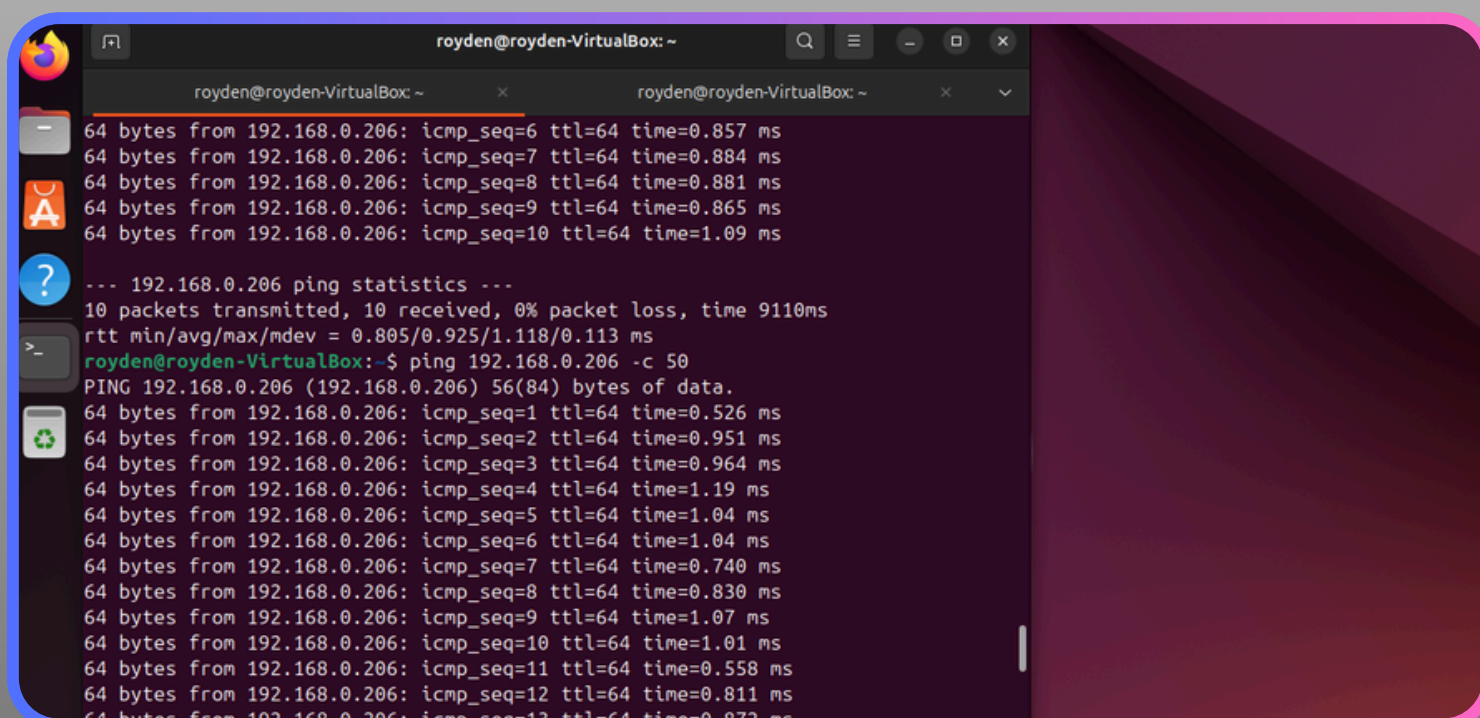


## Step 3

# Attack the server

From the attacker machine generate some malicious traffic. For this lab I have used ping only for demonstration purposes. You can try different types of DOS attacks

While the traffic is generated, the packet count will increment on the Internal server as per the logic built into the code



```
royden@royden-VirtualBox: ~  
64 bytes from 192.168.0.206: icmp_seq=6 ttl=64 time=0.857 ms  
64 bytes from 192.168.0.206: icmp_seq=7 ttl=64 time=0.884 ms  
64 bytes from 192.168.0.206: icmp_seq=8 ttl=64 time=0.881 ms  
64 bytes from 192.168.0.206: icmp_seq=9 ttl=64 time=0.865 ms  
64 bytes from 192.168.0.206: icmp_seq=10 ttl=64 time=1.09 ms  
--- 192.168.0.206 ping statistics ---  
10 packets transmitted, 10 received, 0% packet loss, time 9110ms  
rtt min/avg/max/mdev = 0.805/0.925/1.118/0.113 ms  
royden@royden-VirtualBox:~$ ping 192.168.0.206 -c 50  
PING 192.168.0.206 (192.168.0.206) 56(84) bytes of data:  
64 bytes from 192.168.0.206: icmp_seq=1 ttl=64 time=0.526 ms  
64 bytes from 192.168.0.206: icmp_seq=2 ttl=64 time=0.951 ms  
64 bytes from 192.168.0.206: icmp_seq=3 ttl=64 time=0.964 ms  
64 bytes from 192.168.0.206: icmp_seq=4 ttl=64 time=1.19 ms  
64 bytes from 192.168.0.206: icmp_seq=5 ttl=64 time=1.04 ms  
64 bytes from 192.168.0.206: icmp_seq=6 ttl=64 time=1.04 ms  
64 bytes from 192.168.0.206: icmp_seq=7 ttl=64 time=0.740 ms  
64 bytes from 192.168.0.206: icmp_seq=8 ttl=64 time=0.830 ms  
64 bytes from 192.168.0.206: icmp_seq=9 ttl=64 time=1.07 ms  
64 bytes from 192.168.0.206: icmp_seq=10 ttl=64 time=1.01 ms  
64 bytes from 192.168.0.206: icmp_seq=11 ttl=64 time=0.558 ms  
64 bytes from 192.168.0.206: icmp_seq=12 ttl=64 time=0.811 ms  
64 bytes from 192.168.0.206: icmp_seq=13 ttl=64 time=0.872 ms
```

# Python code flow

Perform a traffic capture

Convert it to CSV using tshark

Find offending IP's based on IP threshold > 40

Convert the data into JSON and send it to AIRIA API

## Step 4: AIRIA AI analysis

01

AIRIA API Receives the JSON

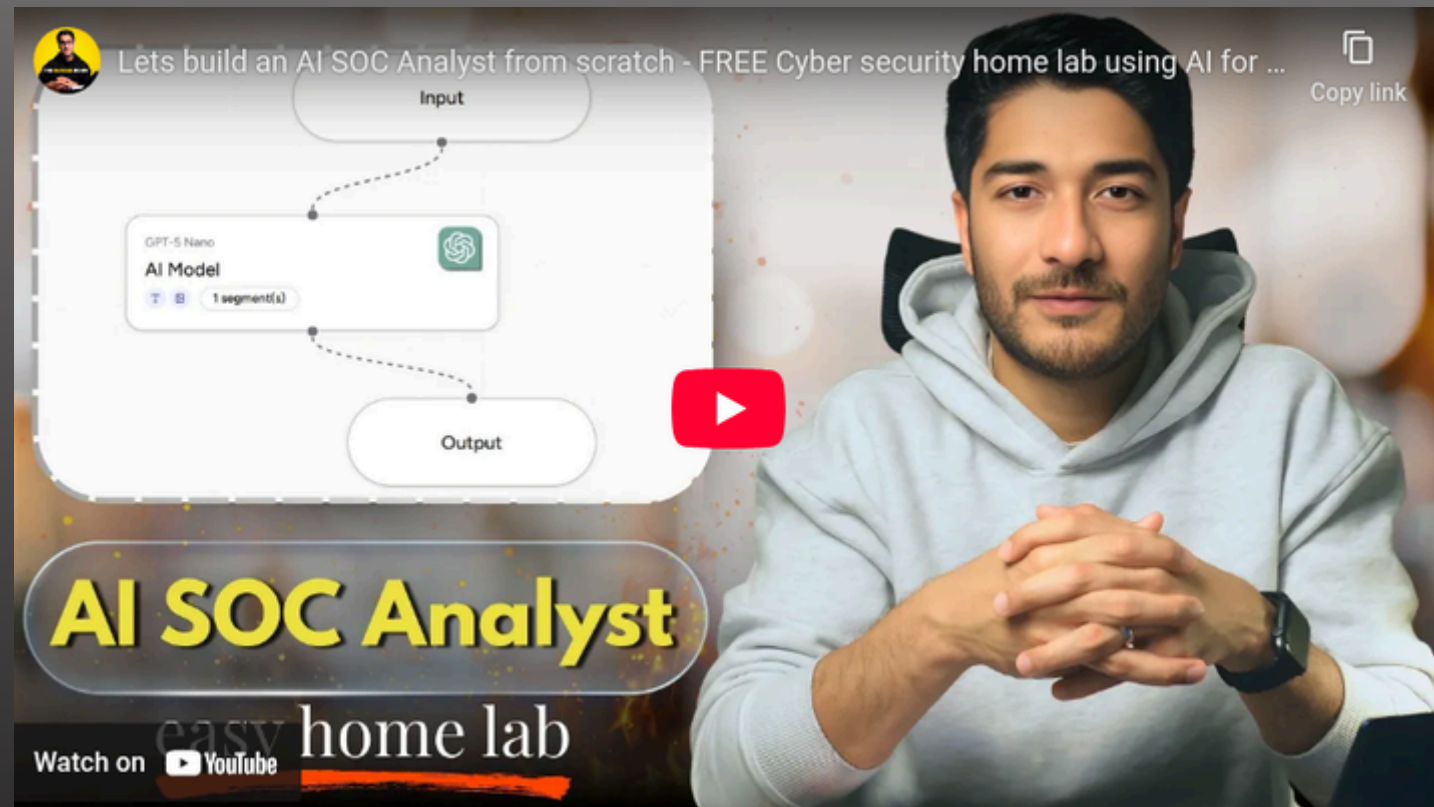
02

AIRIA Agent performs actions  
based on SOC playbook

03

AIRIA Agent sends back the  
analysis to the Internal server

Watch the full lab video here



Follow me on LinkedIn by clicking here



Subscribe to The Social Dork

